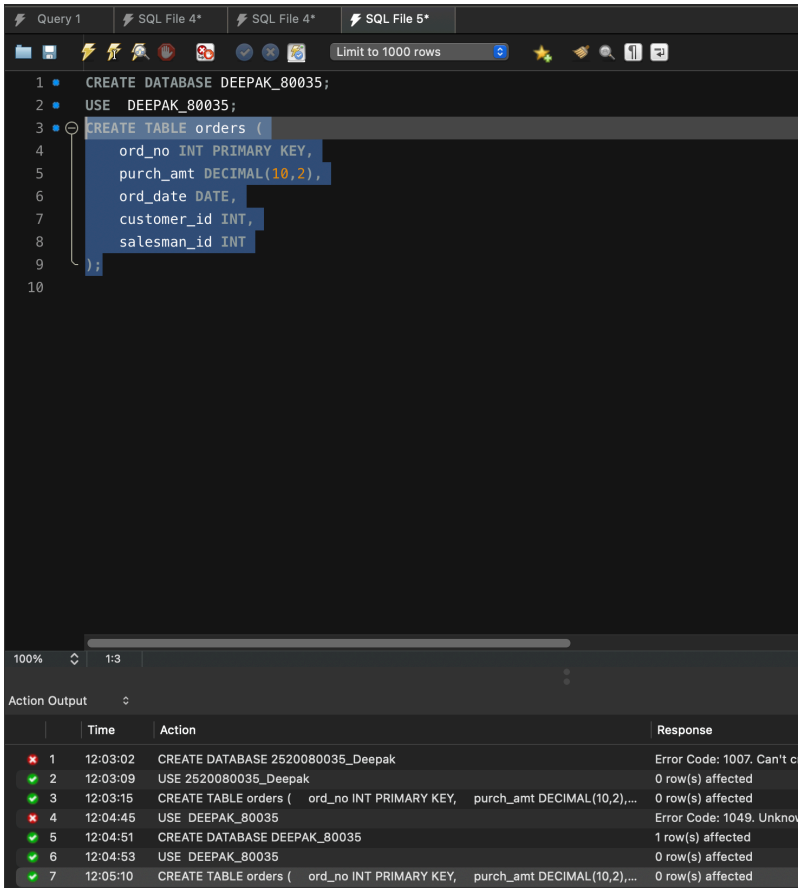
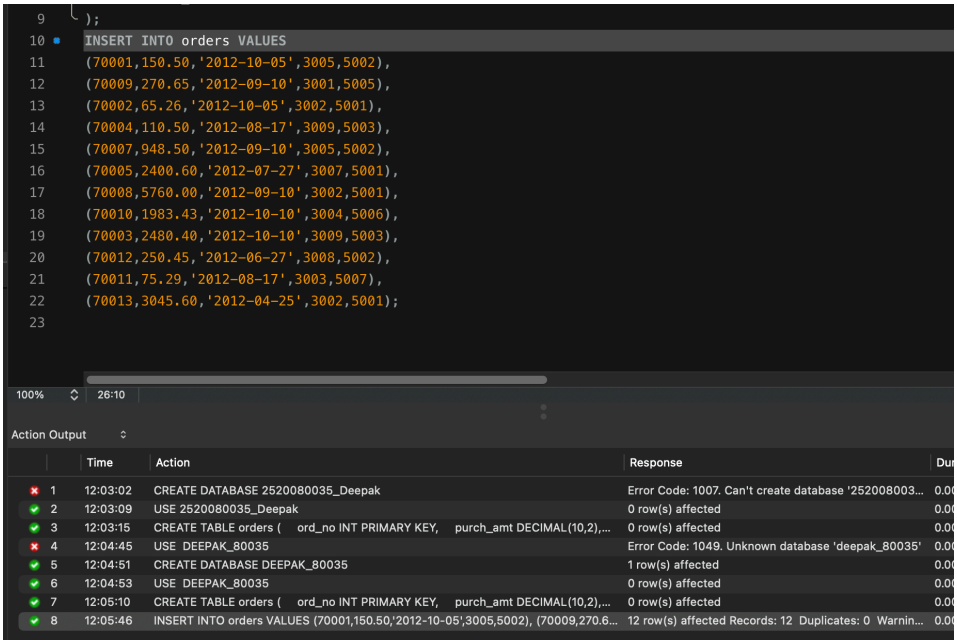
SKILL WEEK-2

Step1: Create Table

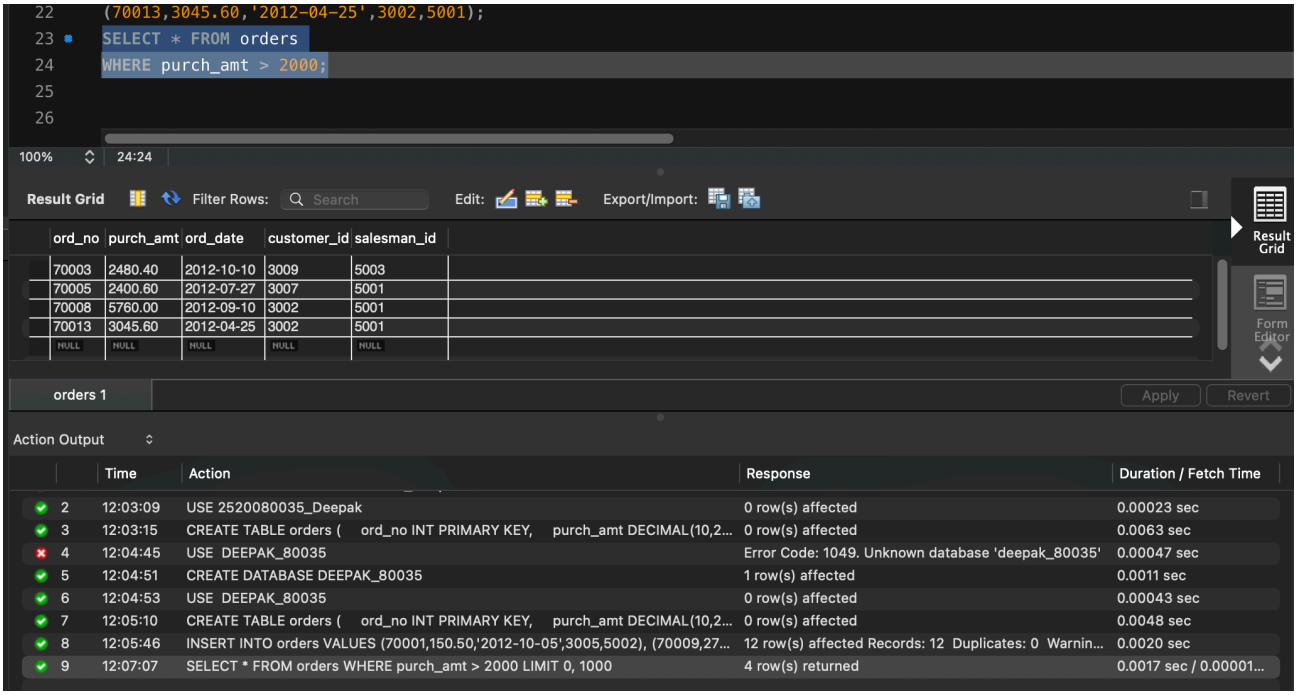


Step2: Insert Data



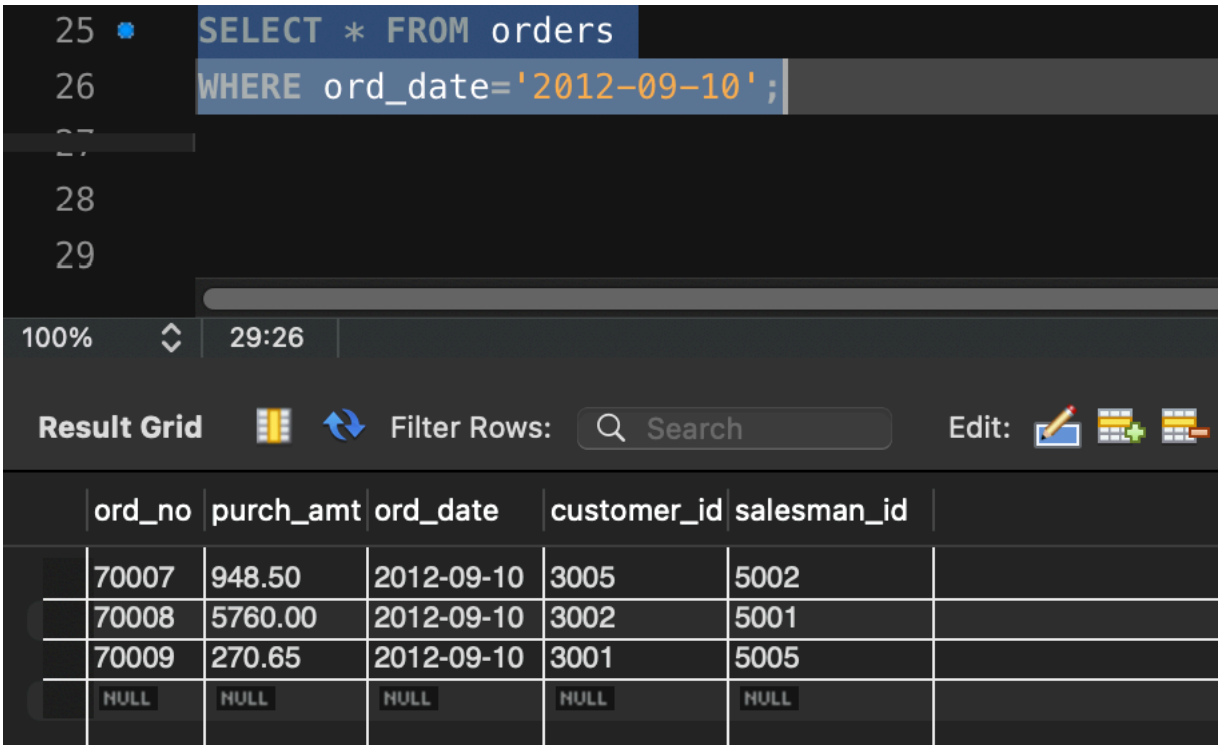
Query1: WHERE CLAUSE

Finance wants all orders whose purchase amount is greater than $2,000



Query2: WHERE CLAUSE

Management wants all orders placed on 2012-09-10.



Query 3 : WHERE CLAUSE

Find all orders handled by salesman 5001

```
27 •  SELECT * FROM orders
28    WHERE salesman_id=5001;
29
```

100% 24:28

Result Grid | Filter Rows: Search | Edit:

ord_no	purch_amt	ord_date	customer_id	salesman_id
70002	65.26	2012-10-05	3002	5001
70005	2400.60	2012-07-27	3007	5001
70008	5760.00	2012-09-10	3002	5001
70013	3045.60	2012-04-25	3002	5001
NULL	NULL	NULL	NULL	NULL

Query 4 : ORDER BY

CEO wants to see orders ranked from highest amount to lowest amount

```
29 •  SELECT *
30    FROM orders
31    ORDER BY purch_amt DESC;
32
33
```

100% 1:29

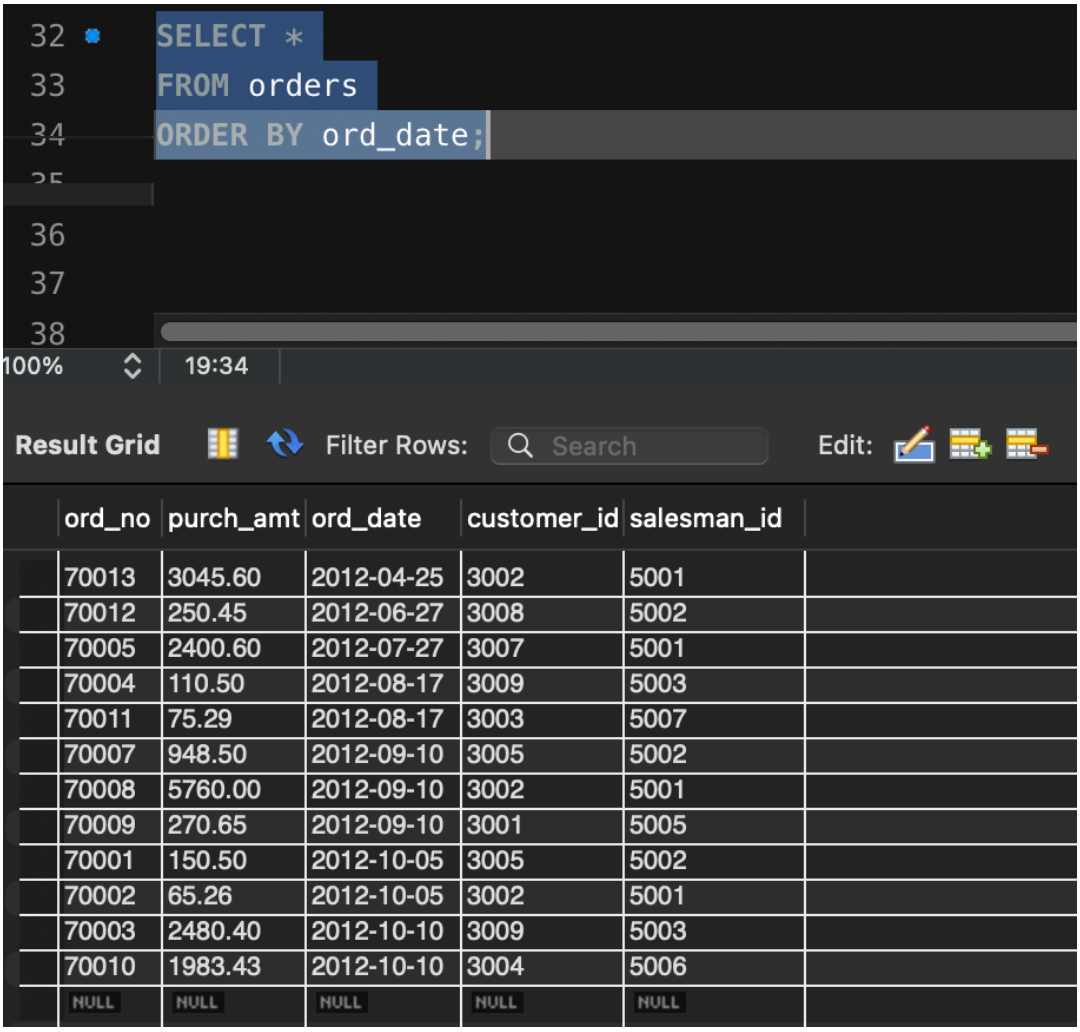
Result Grid | Filter Rows: Search | Edit:

ord_no	purch_amt	ord_date	customer_id	salesman_id
70008	5760.00	2012-09-10	3002	5001
70013	3045.60	2012-04-25	3002	5001
70003	2480.40	2012-10-10	3009	5003
70005	2400.60	2012-07-27	3007	5001
70010	1983.43	2012-10-10	3004	5006
70007	948.50	2012-09-10	3005	5002
70009	270.65	2012-09-10	3001	5005
70012	250.45	2012-06-27	3008	5002
70001	150.50	2012-10-05	3005	5002
70004	110.50	2012-08-17	3009	5003
70011	75.29	2012-08-17	3003	5007
70002	65.26	2012-10-05	3002	5001
NULL	NULL	NULL	NULL	NULL

Query 5 : ORDER BY

Operations team wants oldest orders first.

```
SELECT *
FROM orders
ORDER BY ord_date;
```

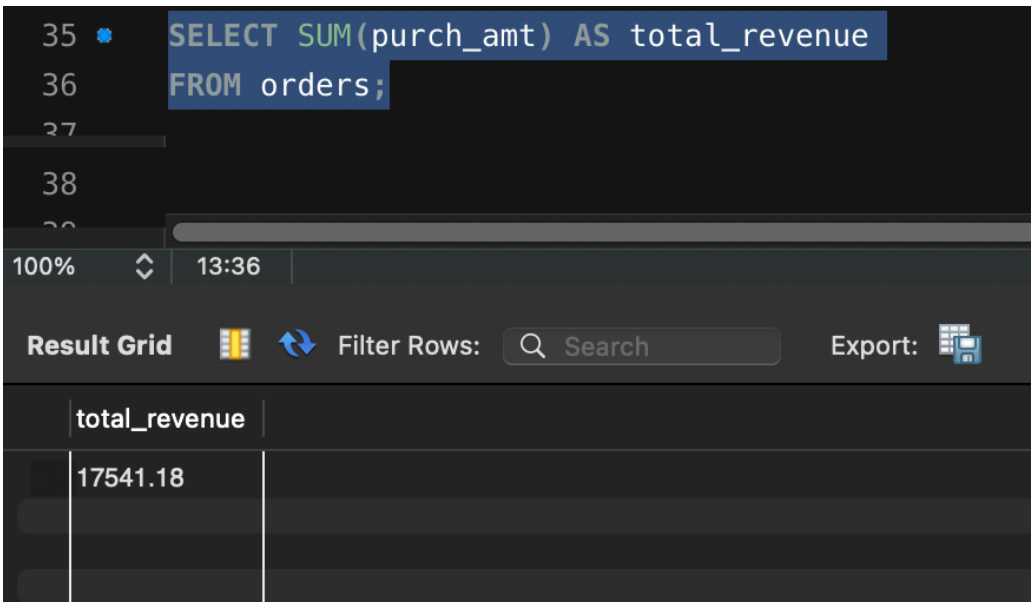


ord_no	purch_amt	ord_date	customer_id	salesman_id
70013	3045.60	2012-04-25	3002	5001
70012	250.45	2012-06-27	3008	5002
70005	2400.60	2012-07-27	3007	5001
70004	110.50	2012-08-17	3009	5003
70011	75.29	2012-08-17	3003	5007
70007	948.50	2012-09-10	3005	5002
70008	5760.00	2012-09-10	3002	5001
70009	270.65	2012-09-10	3001	5005
70001	150.50	2012-10-05	3005	5002
70002	65.26	2012-10-05	3002	5001
70003	2480.40	2012-10-10	3009	5003
70010	1983.43	2012-10-10	3004	5006
NULL	NULL	NULL	NULL	NULL

Query 6 : SUM()

Finance department wants total revenue generated.

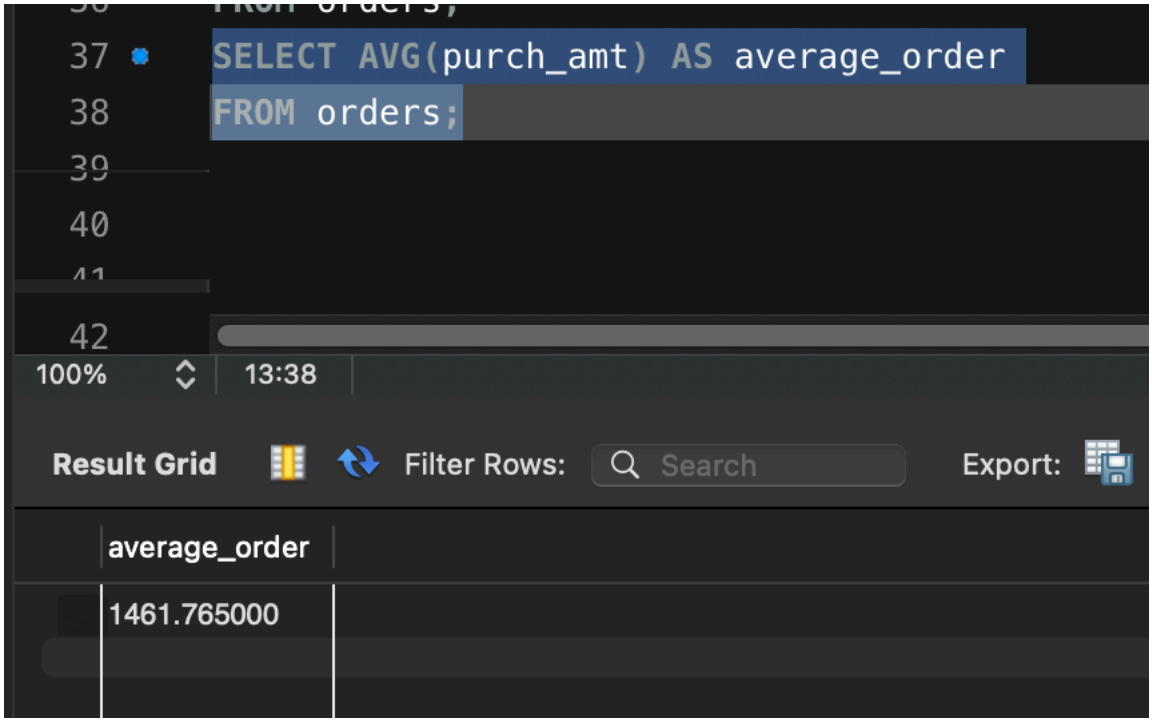
```
SELECT SUM(purch_amt) AS total_revenue
FROM orders;
```



total_revenue
17541.18

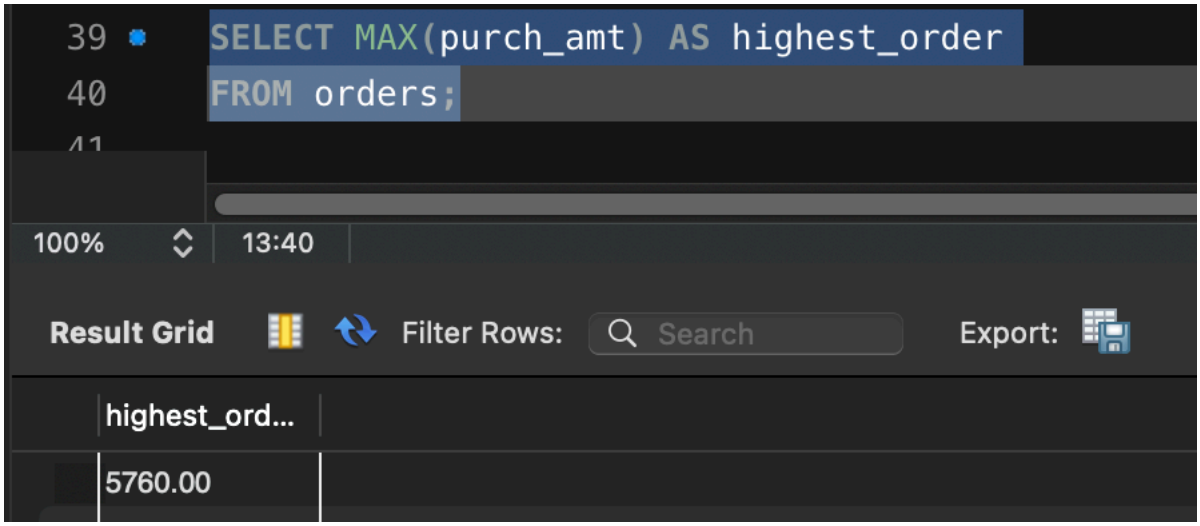
Query 7 : AVG()

Management wants average order value



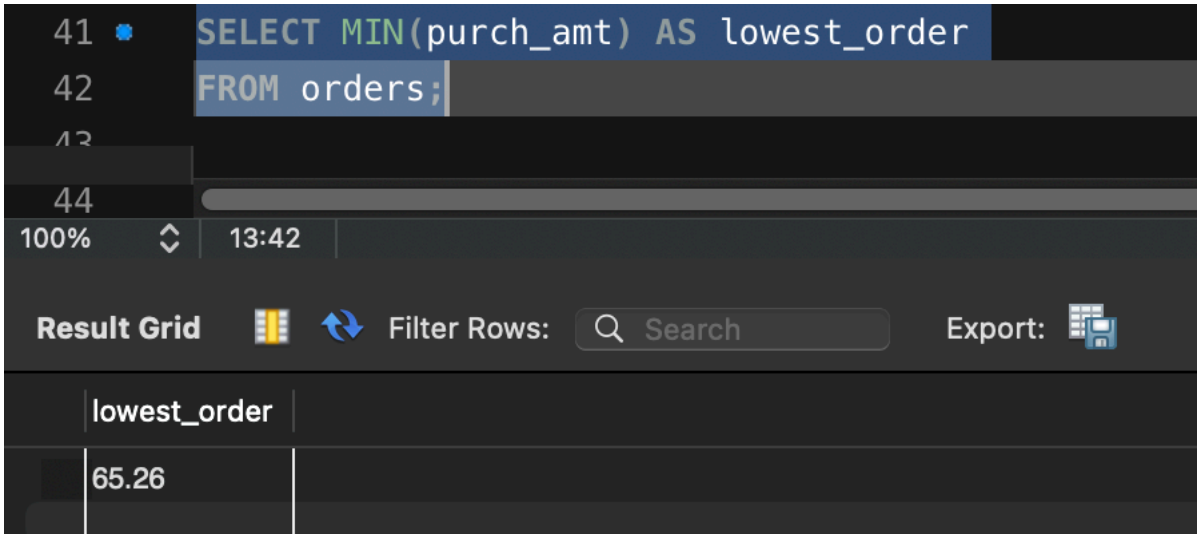
Query 8 : MAX()

Find the highest order amount



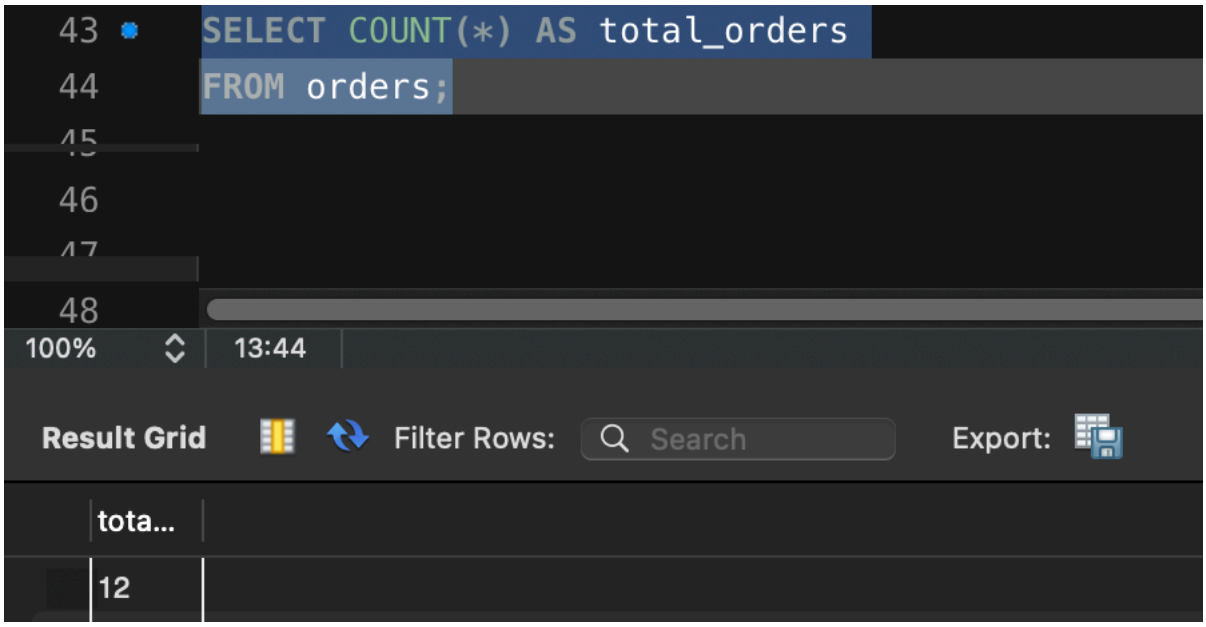
Query 9 : MIN()

Find the lowest order amount



Query 10 : COUNT()

Find total number of orders



Query 11 : GROUP BY

Sales director wants total sales generated by each salesman

```
45  SELECT salesman_id,
46         SUM(purch_amt) AS total_sales
47  FROM orders
48  GROUP BY salesman_id;
49
```

100% 22:48

Result Grid Filter Rows: Search Export:

salesman_id	total_sales
5002	1349.45
5001	11271.46
5003	2590.90
5005	270.65
5006	1983.43
5007	75.29

Query 12 : GROUP BY

Marketing wants total purchases made by each customer

```
49  SELECT customer_id,
50         SUM(purch_amt) AS total_purchase
51  FROM orders
52  GROUP BY customer_id;
```

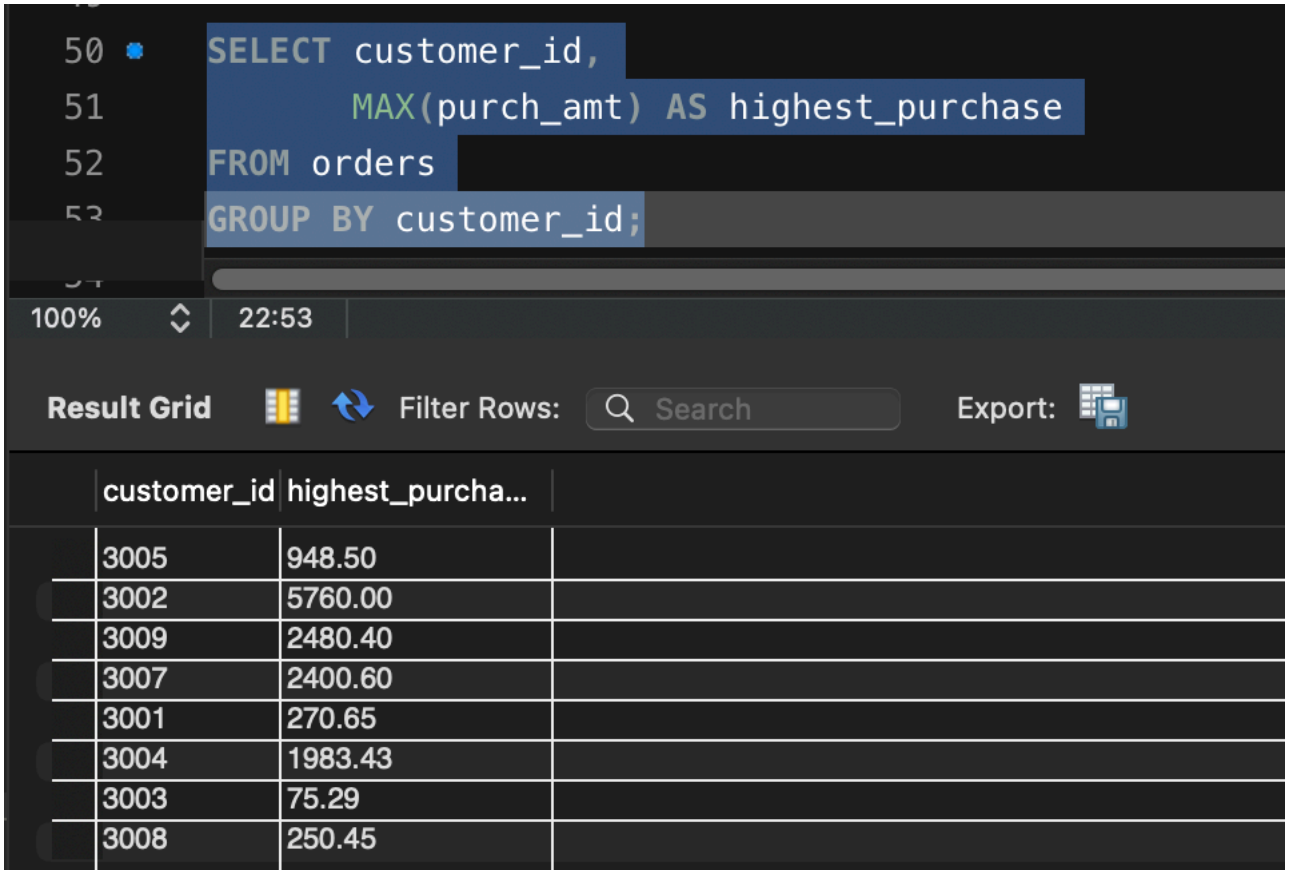
100% 22:52

Result Grid Filter Rows: Search Export:

customer_id	total_purcha...
3005	1099.00
3002	8870.86
3009	2590.90
3007	2400.60
3001	270.65
3004	1983.43
3003	75.29
3008	250.45

Query 13 : GROUP BY + MAX()

Finding the highest order placed by each customer



```
50  SELECT customer_id,
51         MAX(purch_amt) AS highest_purchase
52  FROM orders
53  GROUP BY customer_id;
```

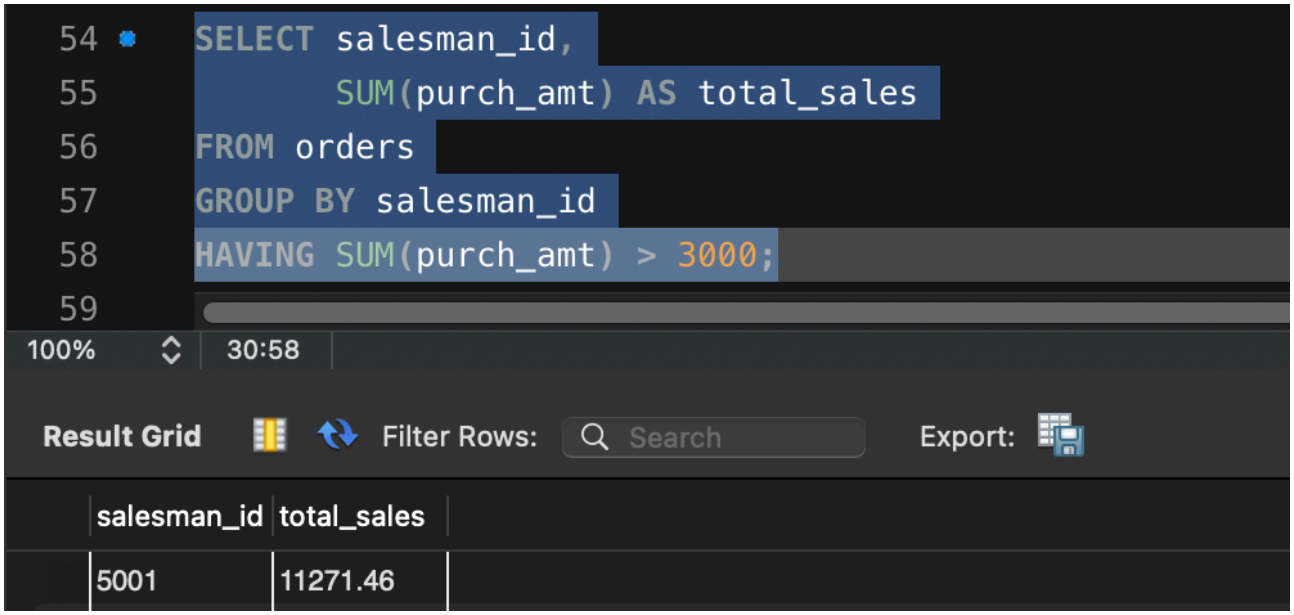
100% 22:53

Result Grid | Filter Rows: Search | Export:

customer_id	highest_purcha...
3005	948.50
3002	5760.00
3009	2480.40
3007	2400.60
3001	270.65
3004	1983.43
3003	75.29
3008	250.45

Query 14 : GROUP BY + HAVING

Company rewards salesmen whose total sales exceeds $3,000



```
54  SELECT salesman_id,
55         SUM(purch_amt) AS total_sales
56  FROM orders
57  GROUP BY salesman_id
58  HAVING SUM(purch_amt) > 3000;
59
```

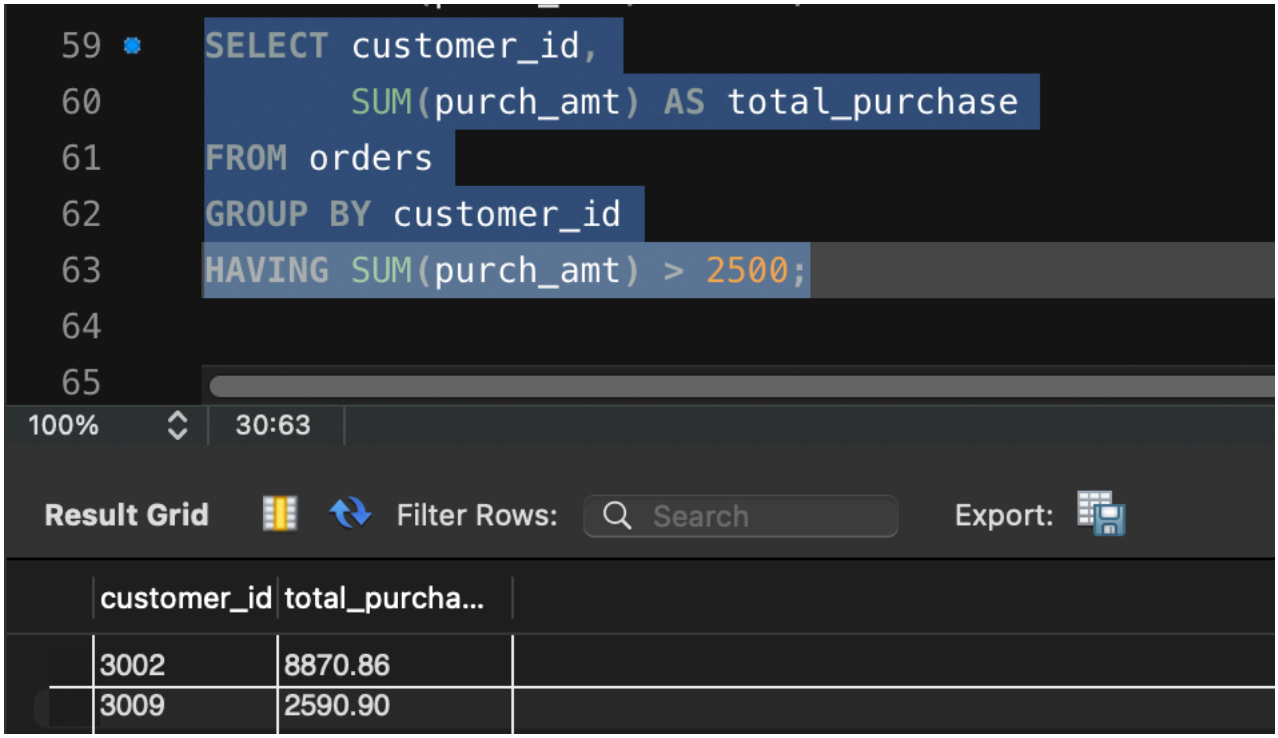
100% 30:58

Result Grid | Filter Rows: Search | Export:

salesman_id	total_sales
5001	11271.46

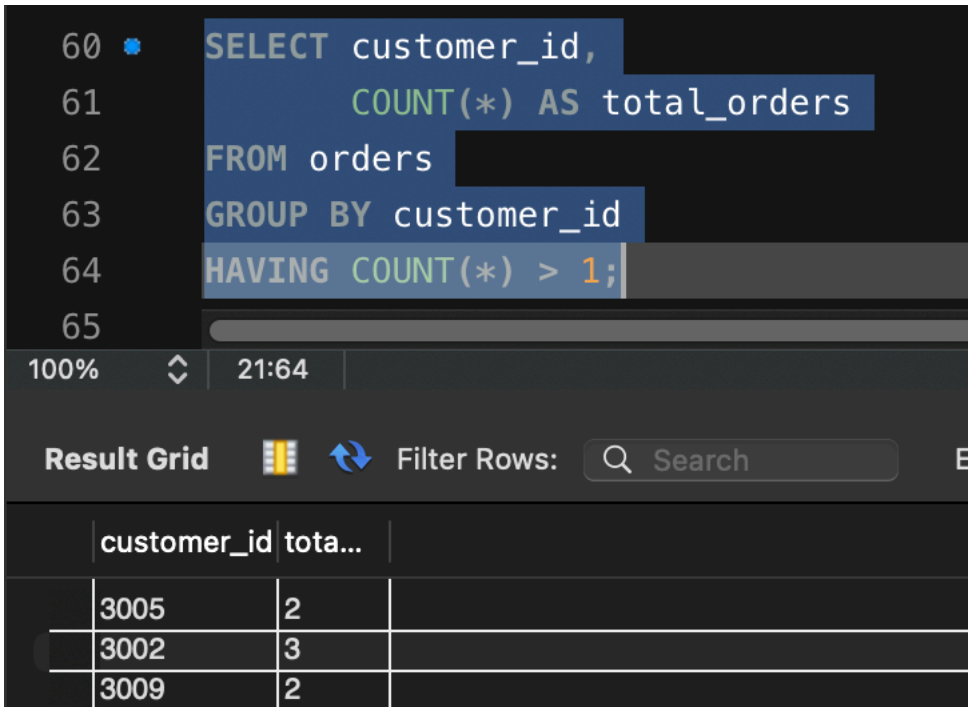
Query 15 : GROUP BY + HAVING

Find customers whose total purchases exceed $2,500



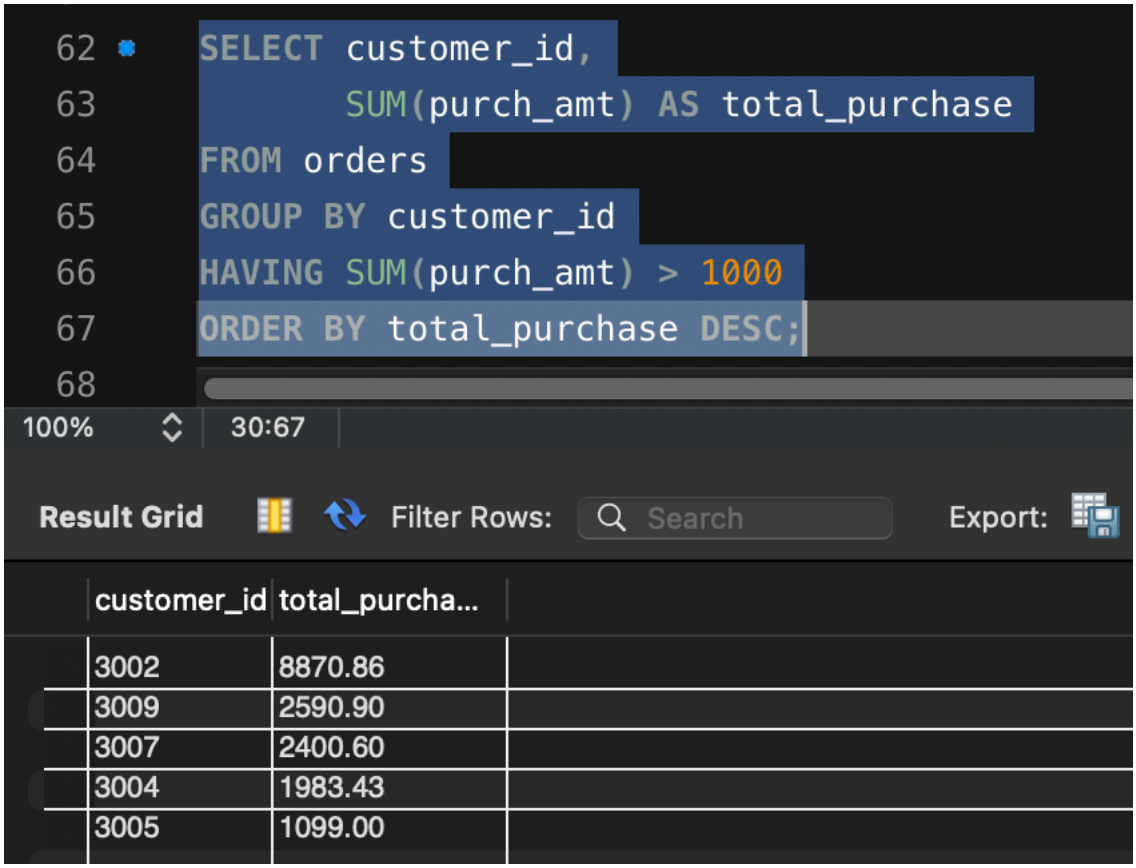
Query 16 : GROUP BY + HAVING

Find customers who placed more than one order



Query 17 : GROUP BY + HAVING + ORDER BY

Management wants top customers ranked by spending



```
62  SELECT customer_id,
63         SUM(purch_amt) AS total_purchase
64  FROM orders
65  GROUP BY customer_id
66  HAVING SUM(purch_amt) > 1000
67  ORDER BY total_purchase DESC;
68
```

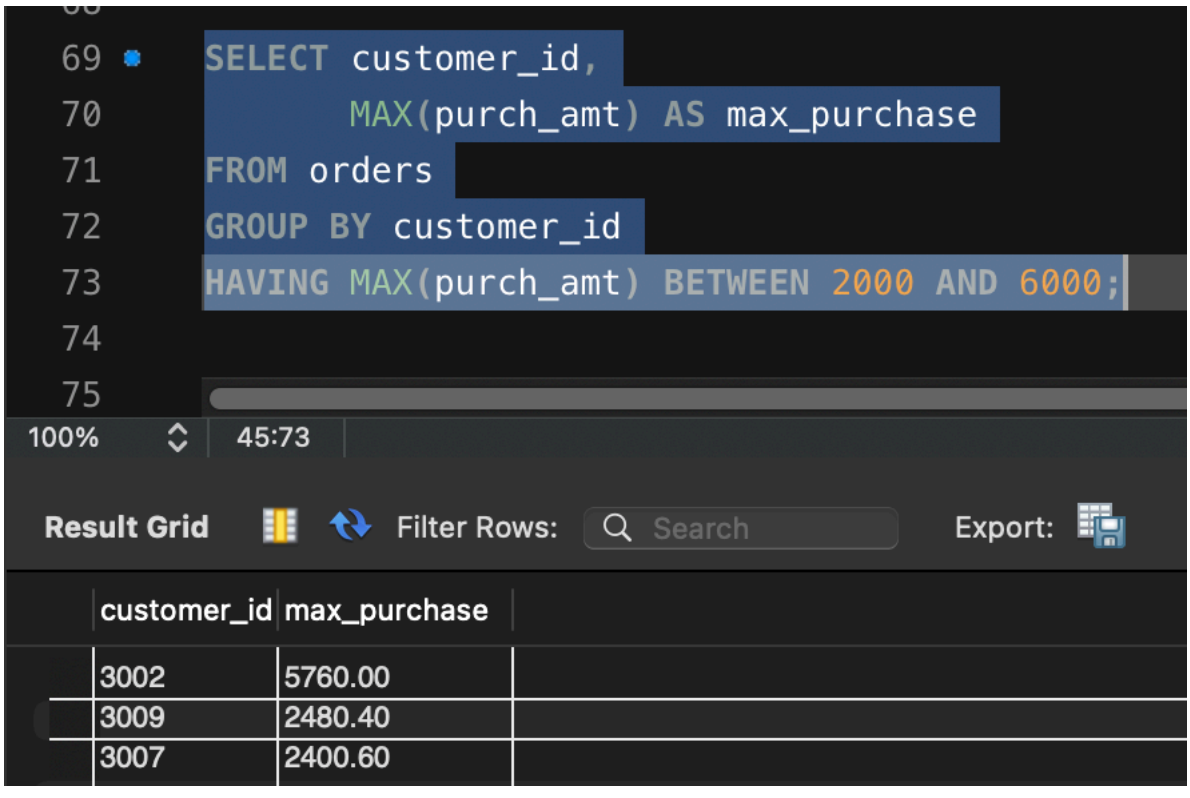
100% 30:67

Result Grid Filter Rows: Search Export:

customer_id	total_purcha...
3002	8870.86
3009	2590.90
3007	2400.60
3004	1983.43
3005	1099.00

Query 18 : BETWEEN + HAVING

Find customers whose maximum purchase is between $2,000 and $6,000.



```
69  SELECT customer_id,
70         MAX(purch_amt) AS max_purchase
71  FROM orders
72  GROUP BY customer_id
73  HAVING MAX(purch_amt) BETWEEN 2000 AND 6000;
74
75
```

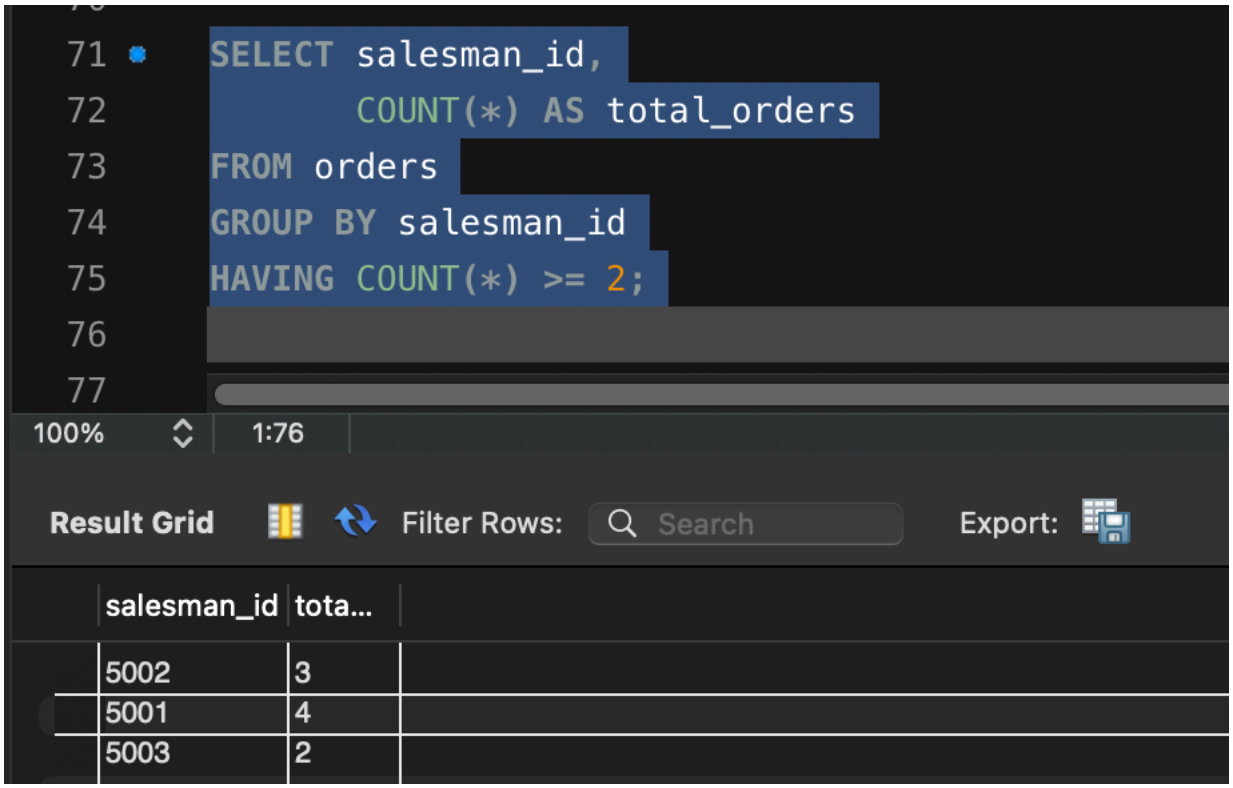
100% 45:73

Result Grid Filter Rows: Search Export:

customer_id	max_purchase
3002	5760.00
3009	2480.40
3007	2400.60

Query 19 : COUNT + HAVING

Find salesmen who handled at least two orders



Query 20 : MAX + GROUP BY + HAVING

Management wants to daily highest purchases above $2000